

Les systèmes à microprocesseur

1. Introduction

Un système à microprocesseur (μp) comprend essentiellement les circuits illustrés à la figure ci-dessous et interconnectés par trois types de bus.

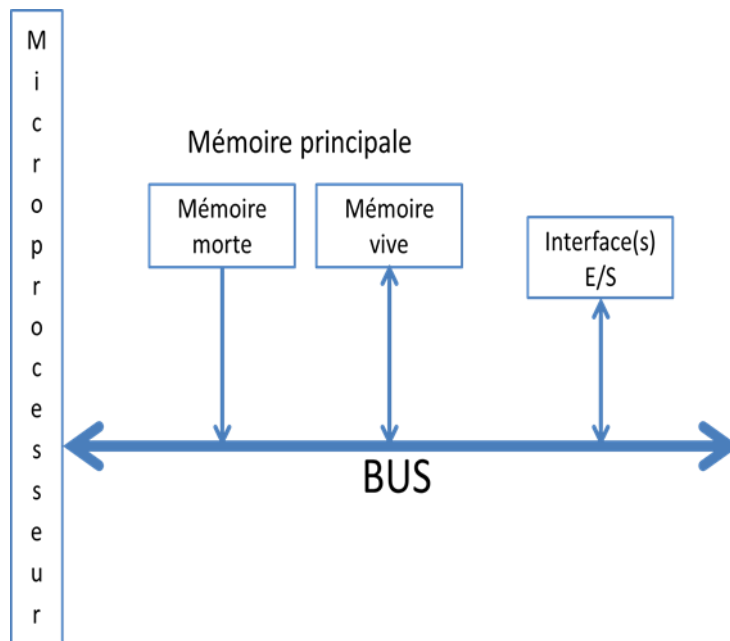


Fig 1 : Architecture de base d'un Système à μ processeur

Bus: ensemble de pistes électroniques qui permettent de véhiculer les données, les adresses ainsi que les signaux de contrôle.

1.1 Rôle des différents circuits

- Le microprocesseur exécute les programmes. Il contient des petites mémoires internes appelées **registres** dont la taille varie selon le type de μp
- Mémoire morte non volatile:
 - contient le programme du constructeur
 - ces mémoires sont actuellement de type flash
- Mémoire vive volatile contient les programmes en cours d'exécution et leurs données
- Les interfaces entrées sorties permettent le contrôle de l'échange des données avec les périphériques

1.2 Les différents types de transferts de données

Le μp est capable de réaliser des opérations de lecture/écriture (READ/WRITE) avec la mémoire et des opérations entrée/sorties (IN/OUT) avec les interfaces entrées sorties (Input/Output).

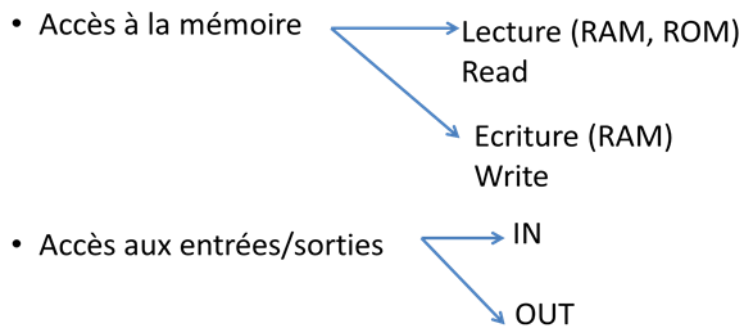


Fig 2 : Echanges de données avec le microprocesseur

2. Stockage des informations

Les mémoires permettent de stocker le code des programmes ou des données généralement sous forme d'octets. Exemple de mémoire de taille 4Koctets:

Adresse en binaire $A_{11}A_{10}A_9A_8A_7A_6A_5A_4A_3A_2A_1A_0$	Contenu $D_7D_6D_5D_4D_3D_2D_1D_0$
0 0 0 0 0 0 0 0 0 0 0 0	1 0 0 1 0 1 1 0
0 0 0 0 0 0 0 0 0 0 0 1	1 1 0 1 0 1 1 0
.	.
1 1 1 1 1 1 1 1 1 1 1 1	1 0 1 0 0 1 0 1

Tableau 1 : Adresses et données d'un circuit mémoire

Taille des bus d'un circuit mémoire

- Avec n bits d'adresses, on peut avoir $N=2^n$ adresses possibles
- On note m le nombre de bits de données

- Pour une RAM Nx m , on a n bits d'adresses et m bits de données

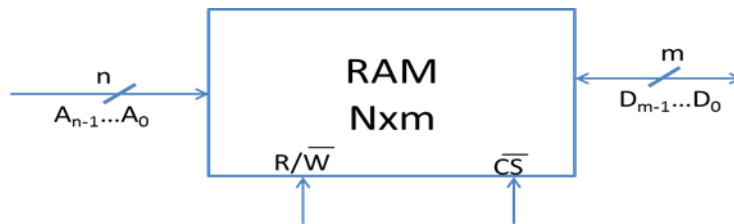


Fig3 : Brochage d'un circuit RAM

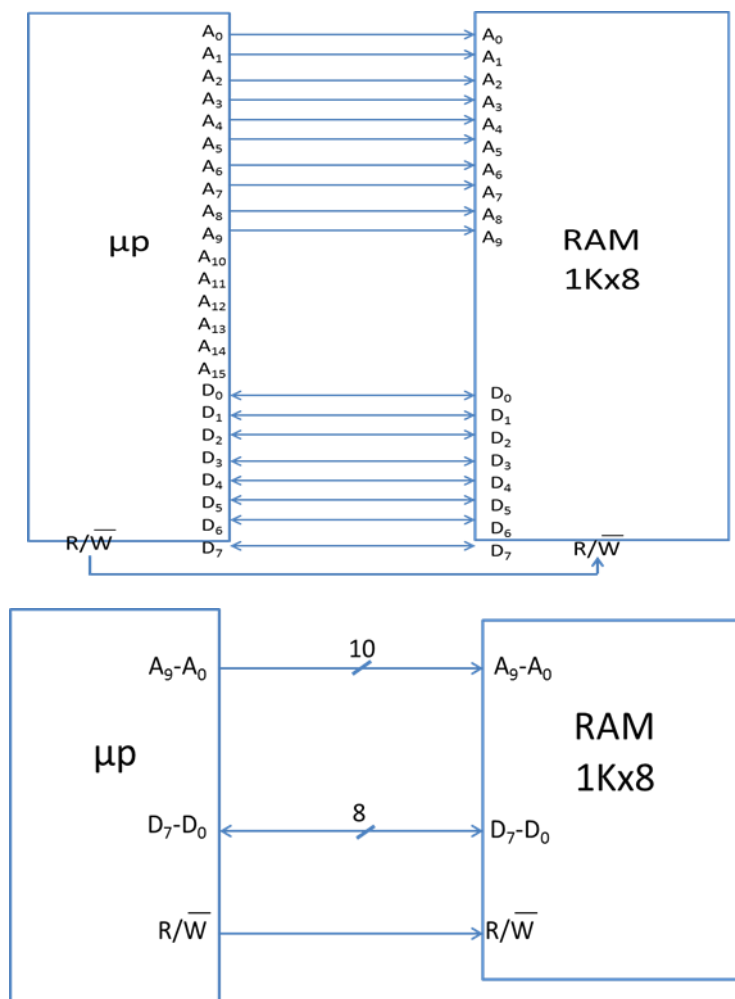


Fig4 : Connexion μp -RAM

Le **bus d'adresses** est unidirectionnel. Il véhicule les bits d'adresse pour sélectionner une case mémoire.

Le **bus de données** est bidirectionnel il permet de véhiculer les données.

Le **bus de contrôle** permet de contrôler les échanges en particulier en précisant le sens de transfert de données.

3. Chronogramme de lecture

Une opération de lecture ou d'écriture de la mémoire suit toujours le même cycle :

1. sélection de l'adresse
2. choix de l'opération à effectuer (R/W)
3. sélection du circuit de la mémoire (CS = 0)
4. lecture ou écriture de la donnée

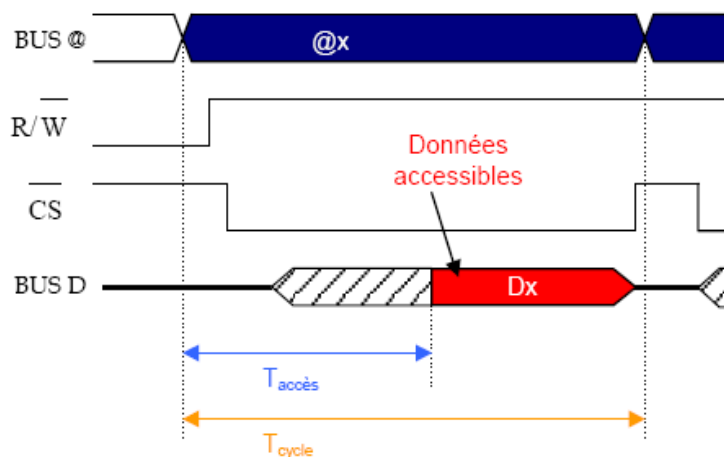


Fig 5 : Chronogramme de lecture

Le **temps d'accès** à la mémoire est le temps qui s'écoule entre l'instant où les adresses sont prêtes et celui où les données sont prêtes.

4. Cartographie de la mémoire (Memory mapping)

Chaque microprocesseur possède une capacité d'adressage qui dépend de la taille de son bus d'adresse. Par exemple un μp qui possède 12 lignes d'adresse peut adresser uniquement 4K adresses. Son espace d'adressage est réparti entre les différents circuits mémoire et entrées/sorties. Chaque circuit aura donc une plage d'adresses qui lui est réservée selon sa taille. La cartographie de la mémoire d'un système à μp est une représentation de l'espace total d'adressage du μp qui indique les différentes plages réservées aux différents circuits du système.

Afin de garantir qu'il n'y'aura pas de conflits d'adresses, on ajoute à la partie matérielle du système un circuit de décodage des adresses qui selon la valeur de l'adresse sortie par le μp active

uniquement le circuit mémoire ou E/S concerné en validant son entrée Chip Select. Les autres circuits restent alors à l'état haute impédance.

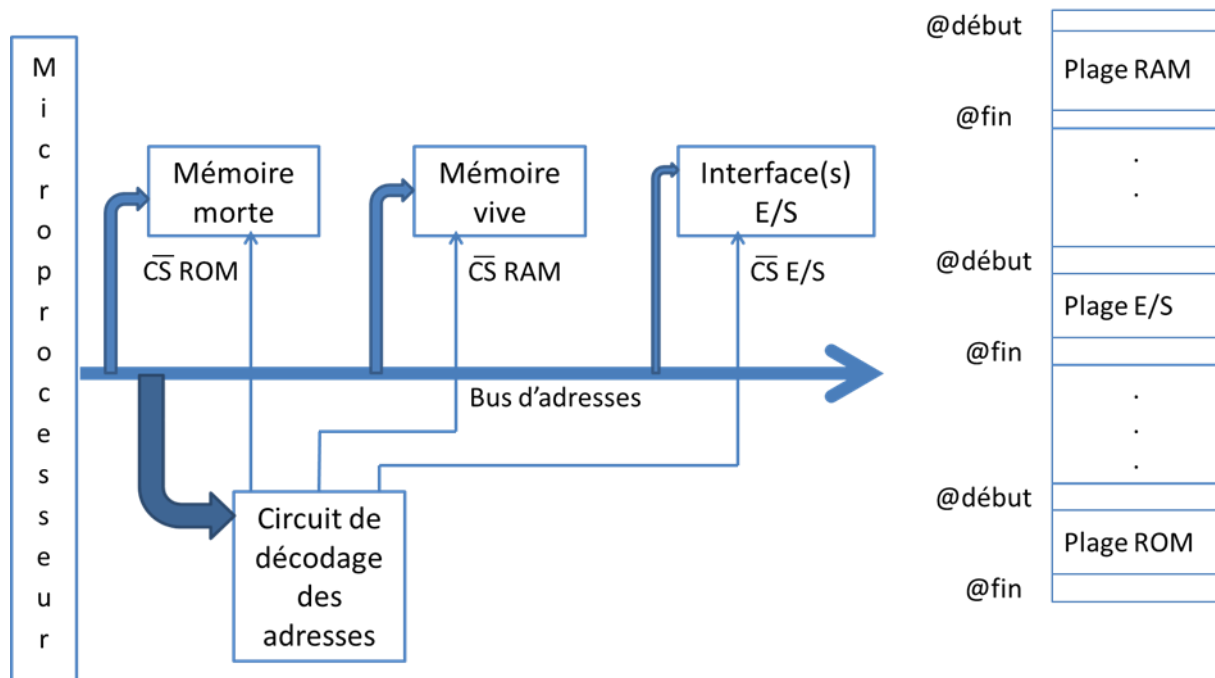


Fig6 : Architecture hardware et mapping de la mémoire

5. Programmation en assembleur

Le langage assembleur est un langage bas niveau. Contrairement aux langages hauts niveau (C, Pascal, basic, Java,...), pour programmer en assembleur, il faut connaître parfaitement l'architecture interne du microprocesseur comme par exemple les noms de registres et leurs tailles.

Le jeu d'instruction (**instruction set**) est alors utilisé pour développer un programme assembleur (**code source**) qui sera assemblé ensuite pour avoir le **code machine** compréhensible par le μp et qui sera stocké dans la mémoire pour être exécuté. Le code source d'un programme assembleur se présente sous la forme de trois colonnes. La première contient les étiquettes, la seconde contient les instructions et la troisième contient les commentaires.

5.1 Les instructions

Une instruction est composée de deux champs:

- Le **code opération** : c'est l'action que le processeur doit accomplir.
- Le **code opérande** : c'est les paramètres de l'action. Le code opérande dépend de l'opération, cela peut être une donnée ou bien une adresse d'un emplacement mémoire.

5.2 Les différents types d'instructions:

- **Transferts de données** : de mémoire à registre, de registre à registre, de registre à mémoire.
- Opérations **arithmétiques** : addition, soustraction, multiplication et division.
- Opérations **logiques** : ET, OU, NON, OU exclusif, etc.
- **Contrôle de séquence** : branchements conditionnels ou non, appel de procédure, etc.
- **Entrées/Sorties**.
- Manipulations **diverses** : décalage, conversion de format, permutation circulaire des bits, échange d'octets, incrémentation, etc.

5.3 Traitement d'une instruction

Le traitement d'une instruction se déroule principalement en trois étapes :

- Recherche (**Fetch**) : Recherche de l'instruction en mémoire.
- Décodage (**Decode**) : Que faut-il faire exemple quel type de calcul
- Exécution (**Execute**) : exécution de l'instruction.

6. Architecture interne du microprocesseur

Comprend deux principales unités:

Unité de commande : est chargé par son décodeur d'instruction de décoder les codes des opérations et de commander les bus du microprocesseur.

Unité d'exécution : contient les registres de données destinées à contenir les opérandes. Elle contient également l'unité arithmétique et logique.

6.1 Les principaux registres du microprocesseur :

Le registre **compteur de programme** (program counter) : contient l'adresse de la prochaine instruction à exécuter.

Le **registre d'état** (status register) : contient les drapeaux d'état (flags) ; ce sont des bits qui renseignent sur le résultat de l'instruction exécutée. Ci-dessous quelques exemples :

Le bit Z indique un résultat nul

Le bit C indique une retenue

Le bit N indique un résultat négatif

Le **registre pointeur de pile** (stack pointer) : contient l'adresse qui pointe le sommet de la pile système. La pile étant une zone de la RAM utilisée par le μp pour sauvegarder généralement de façon temporaire ses registres internes. L'empilement (sauvegarde : écriture dans la pile) se fait par une instruction de type " PUSH". Le dépilement (restauration : lecture à partir de la pile) se fait par une instruction de type " POP". La pile système est de type LIFO (Last In First Out). La pile est aussi utilisée pour sauvegarder le **contexte du μp** lors des appels de sous programme ou des interruptions. Dans ce cas le μp empile le registre PC avant de se brancher au sous programme et le dépile lors du retour du sous programme pour continuer le traitement séquentiel des instructions.

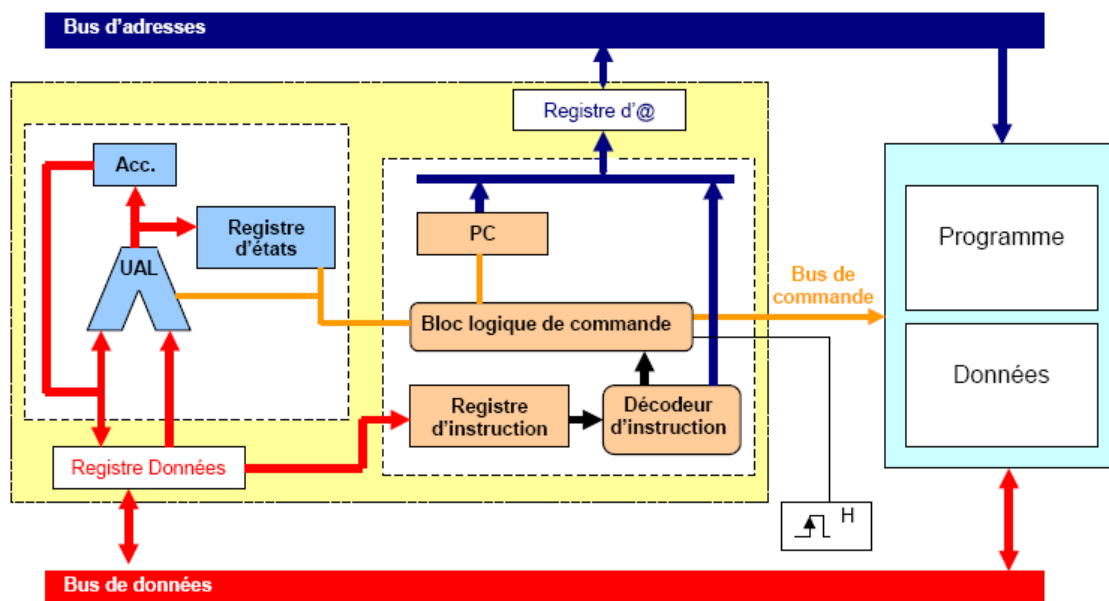


Fig7 : Architecture interne du microprocesseur

7. Les modes d'adressage

On choisit d'utiliser le mode d'adressage qui permet de réaliser l'action souhaité par l'instruction du programme.

7.1 Adressage immédiat

Est généralement utilisé pour charger des données dans les registres internes du microprocesseur. La valeur de l'opérande est alors précisée dans le code de l'instruction.

Exemple1 : `MOVE #H'12,A` ; Charger la valeur 12h dans le registre A du μp

On peut également l'utiliser lors des opérations arithmétiques et logiques.

Exemple2 : `ADD #1,B` ; Ajouter la valeur 1 au registre B

7.2 Adressage étendu

Est généralement utilisé lors des transferts avec la mémoire lors des opérations de type LOAD (lecture) ou STORE (écriture).

Exemple1 : `MOVE H'100,A` ;Charger le registre A avec le contenu de l'adresse 100h.

Exemple2 : `MOVE A,H'102` ; Stocker le contenu du registre à l'adresse 102h.

7.3 Adressage indexé

Utilise les registres d'index pour manipuler généralement les tableaux de données.

Exemple :

MOVE TABLE+{X}, A ; lit une valeur à partir d'une table dans le registre A

7.4 Adressage par registre

Utilisé pour manipuler le contenu d'un registre ou pour réaliser des opérations d'échange ou de calcul en utilisant les registres. Il ne nécessite pas l'accès à la mémoire.

Exemple1 : MOVE A,B ; transférer le contenu de A dans B

Exemple2 : ADD A,B ; additionner A à B et mettre le résultat dans B

Exemple 3 : INC A ; incrémenter A

DEC B ; décrémenter B

CLR X ; charger le registre X par une valeur nulle

8. Echange de données avec les périphériques

Le μ p effectue les échanges avec les périphériques à travers leurs interfaces qui permettent de :

- Adapter les signaux électriques
- Coder les données
- Gérer les erreurs de transmission de données
- Contrôler le flux de données

Les interfaces de périphériques possèdent généralement trois types d'adresses (port d'entrée/sortie) qui doivent figurer dans la cartographie de la mémoire du système.

- **Port de donnée** : pour échanger des données avec le périphérique
- **Port d'état** : permet de connaître l'état du périphérique
- **Port de commande** : permet d'envoyer une commande vers le périphérique.

8.1 Echange par scrutation (polling)

Le microprocesseur interroge l'interface pour savoir si des transferts sont prêts. Tant que des transferts ne sont pas prêts, le microprocesseur attend. L'inconvénient majeur est que le microprocesseur se retrouve souvent en phase d'attente.

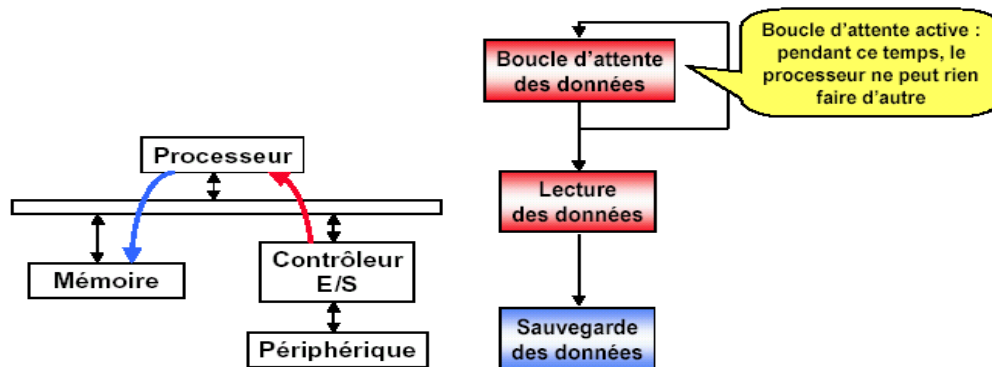


Fig8 : Echange de données par scrutation